

Lab 5: Joystick & DC Motor Control

Engineer: Stanley Etienne / J00974053

Course: ECEL-360

Date: 02-18-26

Objective

This lab's goal was to use a joystick module to control a DC motor's speed and direction using an ESP32. The L293D H-Bridge driver used digital logic signals to control the motor's rotation direction, while the analog position of the joystick was transferred to PWM signals to control motor speed.

Materials

- ESP32 Development Board
- L293D Motor Driver IC
- DC Motor
- Joystick Module
- External 5V Power Supply (for the motor)
- Breadboard
- Jumper wires
- USB cable
- Arduino IDE

Background and Theory

A DC motor cannot be connected directly to a microcontroller because it requires more current than the ESP32 can safely provide. As discussed in the motor driver section,

The L293D H-Bridge driver allows safe control of both motor speed and direction.

Speed Control:

Motor speed is controlled using Pulse Width Modulation (PWM). The ESP32 generates a PWM signal on the Enable1 pin of the L293D. Increasing duty cycle increases motor speed.

Direction Control:

The motor direction is controlled using two digital pins (IN1 and IN2).

- IN1 HIGH, IN2 LOW → Forward rotation
- IN1 LOW, IN2 HIGH → Reverse rotation

Joystick Operation:

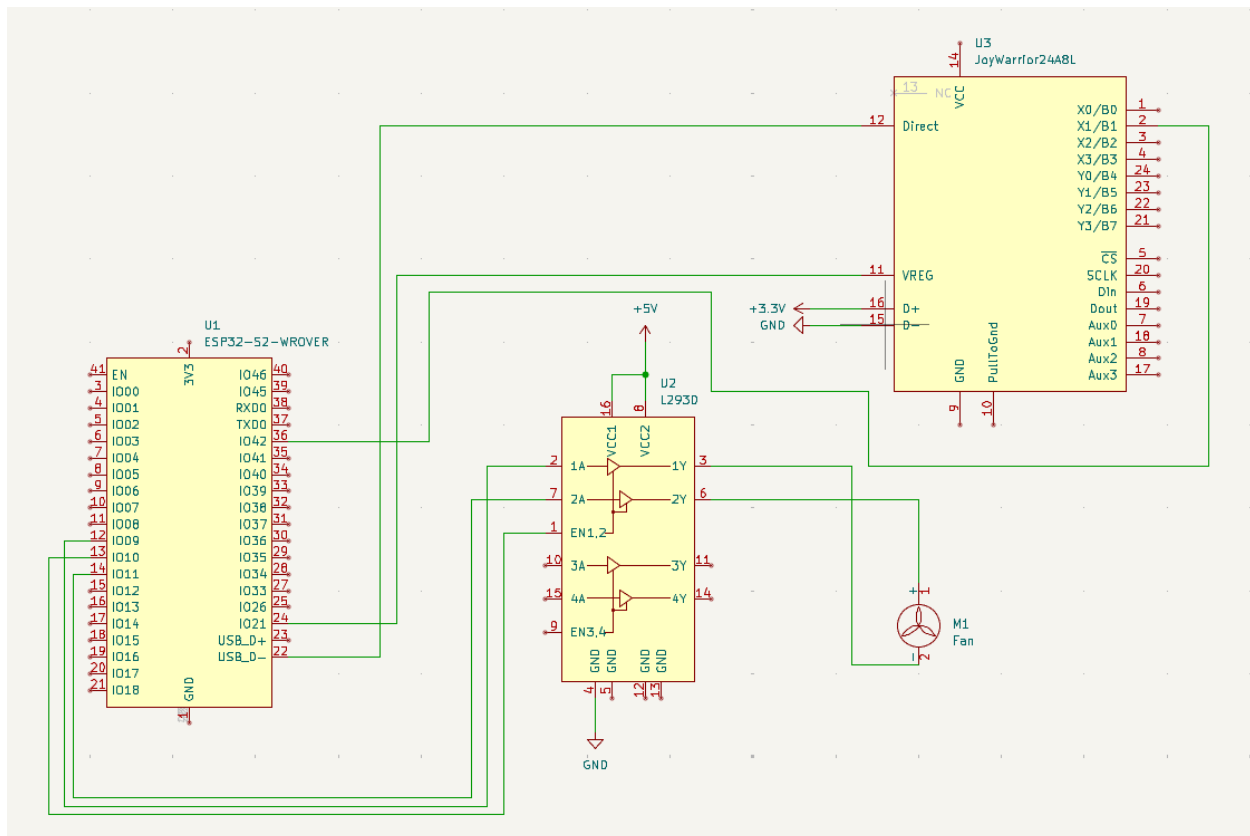
The joystick outputs an analog value between 0 and 4095.

- Center position ≈ 2048
- Values above center → one direction
- Values below center → opposite direction
- Distance from center determines speed

When the joystick is centered, the motor should not spin.

This lab demonstrates how analog input can be translated into proportional motor control using PWM and digital logic.

Wiring Configuration



- GPIO13 → L293D PIN1 (Enable1 – PWM Speed)
- GPIO12 → L293D PIN2 (IN1 – Direction A)
- GPIO14 → L293D PIN7 (IN2 – Direction B)
- L293D OUT1 & OUT2 → DC Motor terminals
- L293D PIN8 → External 5V motor supply
- L293D PIN16 → 5V logic supply
- L293D GND pins → Common Ground
- Joystick X-axis → GPIO34
- Joystick VCC → 3.3V
- Joystick GND → GND

Procedure

1. Connected the L293D motor driver to the ESP32 using the specified GPIO pins.
2. Connected the joystick X-axis output to GPIO34.
3. Connected the DC motor to the L293D output terminals.
4. Supplied external 5V power to the motor driver.
5. Configured PWM on GPIO13.
6. Read analog values from the joystick.
7. Implemented direction control logic based on joystick position.
8. Mapped joystick displacement to PWM duty cycle.
9. Tested forward, reverse, and neutral positions.

Code:

```
const int joyXPin = 34;
const int joyBtnPin = 32;

const int in1Pin = 12;
const int in2Pin = 14;
const int enable1Pin = 13;

const int freq = 1000;
```

```

const int resolution = 11;
const int pwmChannel = 0;

bool motorEnabled = true;
bool lastBtnState = HIGH;

void setup() {
    Serial.begin(115200);

    pinMode(in1Pin, OUTPUT);
    pinMode(in2Pin, OUTPUT);
    pinMode(enable1Pin, OUTPUT);

    pinMode(joyBtnPin, INPUT_PULLUP);

    ledcAttachChannel(enable1Pin, freq, resolution, pwmChannel);
}

void loop() {
    int joyVal = analogRead(joyXPin);
    bool currentBtnState = digitalRead(joyBtnPin);

    // Toggle motor enabled/disabled on button press
    if (lastBtnState == HIGH && currentBtnState == LOW) {
        motorEnabled = !motorEnabled;
        Serial.print("--- MOTOR MODE CHANGED: ");
        Serial.println(motorEnabled ? "ENABLED ---" : "LOCKED/DISABLED ---");
        delay(50);
    }
    lastBtnState = currentBtnState;

    int speed = 0;
    bool direction = true;

    // Dead-zone + mapping logic
    if (!motorEnabled) {
        speed = 0;
    }
    else if (joyVal > 2200) {
        direction = true;
        speed = map(joyVal, 2200, 4095, 0, 2047);
    }
    else if (joyVal < 1900) {
        direction = false;
        speed = map(joyVal, 1900, 0, 0, 2047);
    }
}

```

```

    }
    else {
        speed = 0;
    }

    driveMotor(direction, speed);

    Serial.print("Joy: "); Serial.print(joyVal);
    Serial.print(" | Status: "); Serial.print(motorEnabled ? "ACTIVE" : "LOCKED");
    Serial.print(" | Speed: "); Serial.println(speed);

    delay(15);
}

void driveMotor(boolean dir, int spd) {
    if (dir) {
        digitalWrite(in1Pin, HIGH);
        digitalWrite(in2Pin, LOW);
    } else {
        digitalWrite(in1Pin, LOW);
        digitalWrite(in2Pin, HIGH);
    }
    ledcWrite(enable1Pin, spd);
}

```

Results

The joystick successfully controlled both the **direction** and **speed** of the DC motor. When the joystick was near the center (dead-zone region), the motor remained off as expected. When the joystick was pushed past the upper threshold, the motor rotated in one direction and increased speed proportionally as the joystick was pushed further. When the joystick was pushed below the lower threshold, the motor reversed direction and the speed again increased proportionally to joystick displacement.

The joystick push button (GPIO32) successfully toggled the motor between **ENABLED (ACTIVE)** and **LOCKED (DISABLED)** modes. When locked, the motor stayed off regardless of joystick position. Serial Monitor output confirmed real-time joystick readings, motor status, and PWM speed values.

Conclusion

This lab successfully demonstrated motor control using the ESP32 with a joystick input and an H-bridge motor driver. The joystick's analog X-axis was mapped into PWM output (0–

2047 with 11-bit resolution) to regulate motor speed, while digital outputs controlled motor direction using the L293D driver. A dead-zone ensured stable behavior when the joystick was centered, preventing unintended motion. Additionally, the joystick button feature provided an effective method to enable/disable the motor during operation. Overall, this lab reinforced key embedded systems concepts including analog-to-PWM mapping, direction logic, and safe motor driving using a motor driver circuit.